COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure and Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	07/17/26	**DATE SUBMITTED:**	7/17/26
NAME:	Macasaet, John David C.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. _2.1__ Arrays, Pointers and Dynamic Memory Allocation

1. PROCEDURE OUTPUT

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
       std::string studentName;
       int studentAge;
    public:

       //CONSTRUCTOR
       Student(std::string newName = "John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." <<std::endl;
       };

       //DECONSTRUCTOR
       ~Student(){
        std::cout << "Destructor Called." <<std::endl;
       }

       //COPY CONSTRUCTOR
       Student(const Student &copyStudent){
        std::cout <<"Copy Constructor Called." <<std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
       }

       //DISPLAY ATTRIBUTES
       void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
       }
};

int main() {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

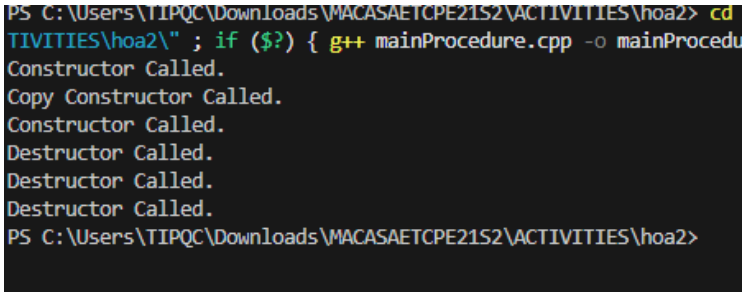
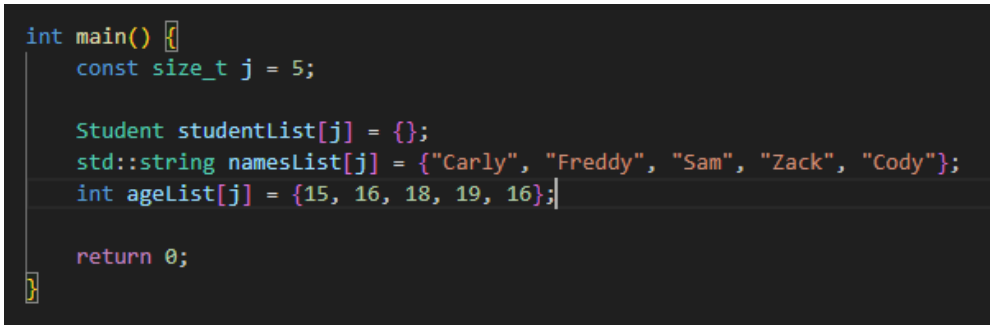
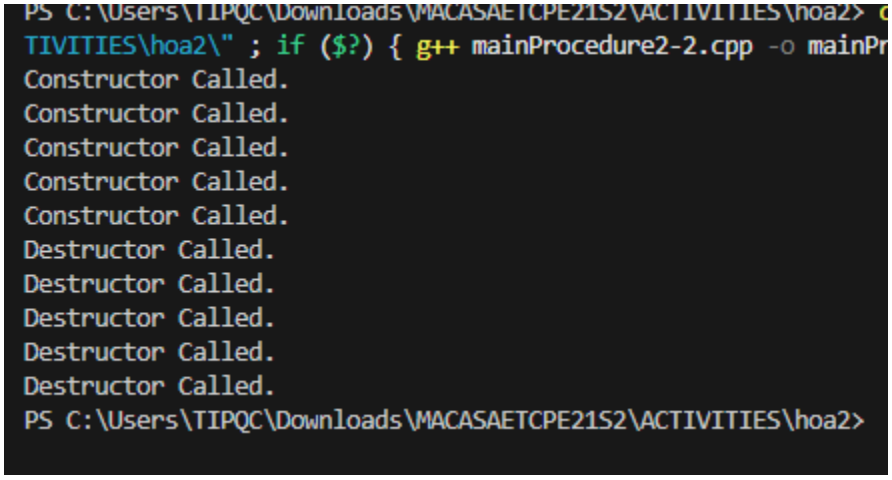
	
Observation	For student 1, the placeholder values which are “John Doe” and “18” were changed into “Roman” and “28” using the constructor. For student 2, using the copy constructor, student 2 was able to copy the values of student 1. In line 38, student 3 values were just the placeholder values because it did not put its own values or copied the values of student 1.

Table 2-1. Initial Driver Program

Screenshot	 

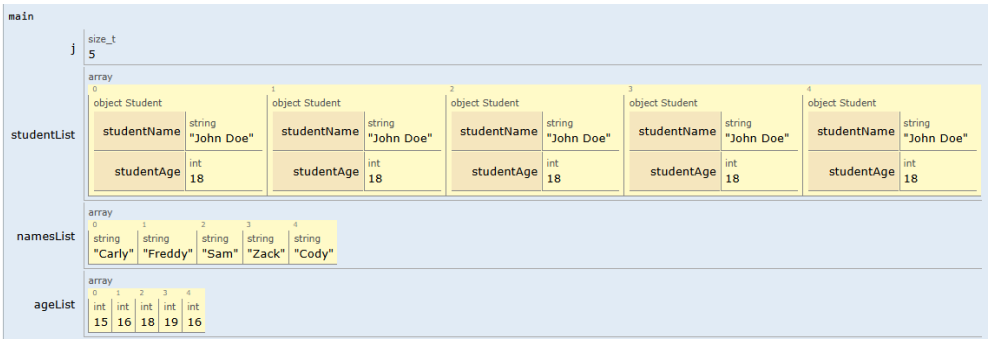
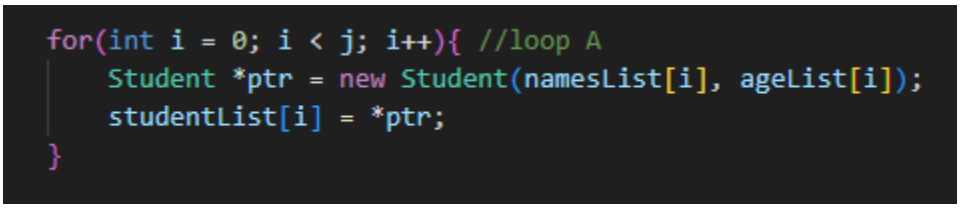
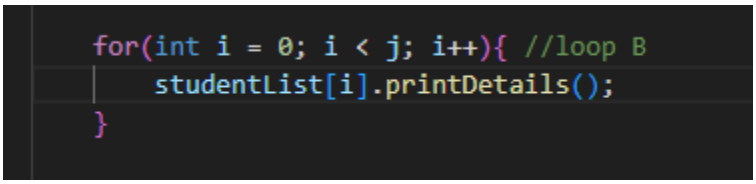
	
Observation	An array with the name studentList with a size of 5 was created and it stored the values in the Student class. The values in namesList and ageList were not stored inside studentList because it did not call the Student class.

Table 2-2. Modified Driver Program with Student Lists

Loop A	
Observation	In this we showcased dynamic memory allocation. The values of namesList and ageList became the values of Student class by allocating memory on a heap for the values inside those two arrays. In studentList[i] = *ptr, *ptr deferences ptr which mean the values inside the address were put inside the studentList array.
Loop B	
Observation	Loop B individually prints each name and age to the studentList array. This replaces those placeholder values “John Doe” and “18”.

```
for(int i = 0; i < j; i++){ //loop A
    Student *ptr = new Student(namesList[i], ageList[i]);
    studentList[i] = *ptr;
}
```

```
for(int i = 0; i < j; i++){ //loop B
    studentList[i].printDetails();
}
```

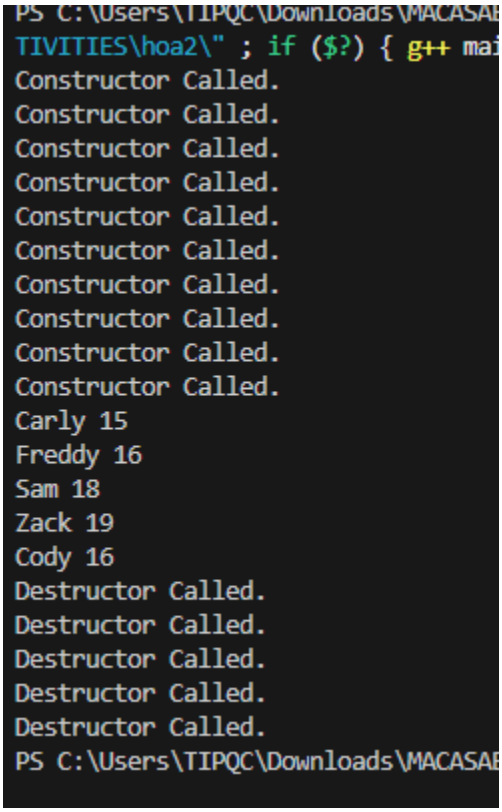
Output	
Observation	Everytime “Constructor Called” is printed that means the values were placed inside the Student class. The names and ages seen in the terminal are the result of printing the studentList array that now contains the given values that replaced the placeholder values in Student class. After this, the destructor was called to uninitialize the Student constructor.

Table 2-3. Final Driver Program

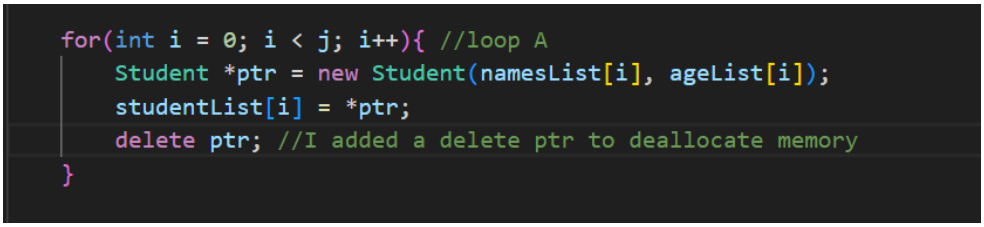
Modifications	
Observation	I added the delete ptr to deallocate the memory from the heap to prevent memory leaks.

Table 2-4. Modifications/Corrections Necessary

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include <iostream>
#include <string.h>

class Fruits{
    private:
        std::string fruitName;
        int fruitPrice;
        int fruitQty;
    public:
        //constructor
        Fruits(std::string newFName = "Blank", int newFPrice = 0, int newFQty = 0){
            fruitName = std::move(newFName);
            fruitPrice = newFPrice;
            fruitQty = newFQty;
            std::cout << "Constructor Called." <<std::endl;
        };

        //deconstructor
        ~Fruits(){
         std::cout << "Destructor Called." <<std::endl;
        }

        //copy constructor
        Fruits(const Fruits &copyFruits){
         std::cout <<"Copy Constructor Called." <<std::endl;
         fruitName = copyFruits.fruitName;
         fruitPrice = copyFruits.fruitPrice;
         fruitQty = copyFruits.fruitQty;
        }
```

```
30        //copy assignment
31        Fruits& operator=(const Fruits& copyFruits) {
32         if (this == &copyFruits) {
33             return *this;
34         }
35
36         this->fruitName = copyFruits.fruitName;
37         this->fruitPrice = copyFruits.fruitPrice;
38         this->fruitQty = copyFruits.fruitQty;
39
40         return *this;
41      }
42        //display attributes
43        void printDetails(){
44         std::cout << this->fruitName << "| Price: " << this->fruitPrice << " Qty: " << this->fruitQty << std::endl;
45        }
46        //sum of fruits prices
47         static int fpriceTotal(const int fruitP[], int size){
48          int totalFSum = 0;
49          for(int i = 0; i < size; i++){
50            totalFSum += fruitP[i];
51          }
52          return totalFSum;
53         }
54    };
55
```

```
55
56    class Vegetables{
57        private:
58            std::string vegName;
59            int vegPrice;
60            int vegQty;
61        public:
62            //constructor
63            Vegetables(std::string newVName = "Blank", int newVPrice = 0, int newVQty = 0){
64                vegName = std::move(newVName);
65                vegPrice = newVPrice;
66                vegQty = newVQty;
67                std::cout << "Constructor Called." <<std::endl;
68            };
69
70            //deconstructor
71            ~Vegetables(){
72             std::cout << "Destructor Called." <<std::endl;
73            }
74
75            //copy constructor
76            Vegetables(const Vegetables &copyVegetables){
77             std::cout <<"Copy Constructor Called." <<std::endl;
78             vegName = copyVegetables.vegName;
79             vegPrice = copyVegetables.vegPrice;
80             vegQty = copyVegetables.vegQty;
81            }
```

```
82          //copy assignment
83          Vegetables& operator=(const Vegetables& copyVegetables) {
84           if (this == &copyVegetables) {
85               return *this;
86           }
87
88           this->vegName = copyVegetables.vegName;
89           this->vegPrice = copyVegetables.vegPrice;
90           this->vegQty = copyVegetables.vegQty;
91
92           return *this;
93       }
94
95          //display attributes
96          void printDetails(){
97           std::cout << this->vegName << "| Price: " << this->vegPrice << " Qty: " << this->vegQty << std::endl;
98          }
99          //sum of vegetable prices
100          static int vpriceTotal(const int vegP[], int size){
101            int totalVSum = 0;
102            for(int i = 0; i < size; i++){
103              totalVSum += vegP[i];
104            }
105            return totalVSum;
106          }
107
108  };
```

```
109  //Total price sum of fruits and vegetables
110  int totalSum(int a, int b){
111      int totSum = a + b;
112      return totSum;
113  }
114
115
116  int main(){
117      std::string* GroceryList = new std::string[4]{"Apple", "Banana", "Broccoli", "Lettuce"};
118      std::string fnameList[2];
119      std::string vnameList[2];
120
121      //puts Apple and Banana to fnameList
122      for(int i = 0; i < 2; i++){
123         fnameList[i] = GroceryList[i];
124      }
125      //puts Broccoli and Lettuce in fnameList
126      for(int i = 0; i < 2; i++){
127          vnameList[i] = GroceryList[i+2];
128      }
129      const size_t j = 2;
130      int fpriceList[j] = {10, 10};
131      int fqtyList[j] = {7, 8};
132      int newfpriceList[j] = {};
133      //multiple price to quantity
134      for(int i = 0; i < 2; i++){
135          newfpriceList[i] = fpriceList[i] * fqtyList[i];
136      }
```

```
137
138     int vpriceList[j] = {60, 50};
139     int vqtyList[j] = {12, 10};
140     int newvpriceList[j] = {};
141     for(int i = 0; i < 2; i++){
142         newvpriceList[i] = vpriceList[i] * vqtyList[i];
143     }
144
145     Fruits fruitDetails[j] = {};
146     Vegetables vegDetails[j] = {};
147     for(int i = 0; i < j; i++){
148         Fruits *ptr = new Fruits(fnameList[i], fpriceList[i], fqtyList[i]);
149         fruitDetails[i] = *ptr;
150         delete ptr;
151     }
152
153     for(int i = 0; i < j; i++){
154         Vegetables *ptr = new Vegetables(vnameList[i], vpriceList[i], vqtyList[i]);
155         vegDetails[i] = *ptr;
156         delete ptr;
157     }
```

```
158
159     std::cout << "Fruits:\n";
160     for(int i = 0; i < j; i++){
161         fruitDetails[i].printDetails();
162     }
163     std::cout << "\nVegetables:\n";
164     for(int i = 0; i < j; i++){
165         vegDetails[i].printDetails();
166     }
167     int totalFruitP = Fruits::fpriceTotal(newfpriceList, j);
168     std::cout << "\nTotal Price of Fruits: " << totalFruitP <<std::endl;
169
170     int totalVegP = Vegetables::vpriceTotal(newvpriceList, j);
171     std::cout << "Total Price of Vegetables: " << totalVegP <<std::endl;
172
173     std::cout<<"Total Price: " << totalSum(totalVegP, totalFruitP)<<std::endl;
174
175     GroceryList[3].clear();
176     //to make sure that lettuce has been removed
177     if(GroceryList[3].empty()){
178         std::cout<< "Lettuce has been deleted\n";
179     }
180     //deallocate memory
181     delete[] GroceryList;
182
183
184     return 0;
185 }
```

```
PS C:\Users\LENOVO\Downloads\MACASAETCPE21S2\ACTIV
\" ; if ($?) { g++ supplementaryact.cpp -o supplem
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Fruits:
Apple| Price: 10 Qty: 7
Banana| Price: 10 Qty: 8

Vegetables:
Broccoli| Price: 60 Qty: 12
Lettuce| Price: 50 Qty: 10

Total Price of Fruits: 150
Total Price of Vegetables: 1220
Total Price: 1370
Lettuce has been deleted
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
PS C:\Users\LENOVO\Downloads\MACASAETCPE21S2\ACTIV
```

CONCLUSION: This activity taught me a lot of useful things like the different operators which are Constructor, Copy Constructor, Copy Assignment, Destructor. Through this I knew when to use them and what is the proper syntax when creating and calling them. Pointers were also a significant part of this activity and it enabled me to do dynamic memory allocation which you can see in the lines of code that has 'new' in it. This whole activity was challenging but fun at the same time and it took me a lot of time to finish. I had to search or ask AI to explain to me what is the purpose of some lines of code was because there were some things I was confused about.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: Google Gemini

Purpose of Use:

Examples: I used it to generate ideas, explain certain concepts to me, and also for code assistance.

Extent of Use:

When I faced a problem, I asked AI to give me clues on how to solve the problem and I specifically asked it to not provide me with the correct code. I also asked it for correct syntax to avoid errors.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.